

可搜索对称加密：改进的定义和高效的构造

摘要

本论文提出了两种新的SSE解决方案，具有以下特点：

1. **效率更高**：这两种解决方案比所有以前的常数轮次方案都更高效，特别是在每个返回文档的服务器工作量上，是常数级别，而不是线性与数据量相关。
2. **更强的安全性**：这两种方案提供比以前常数轮次方案更强的安全性。文中还指出了以往SSE安全性概念中的一些隐蔽问题，并展示了如何避免这些问题。第二个方案还实现了所谓的**自适应SSE安全性**，即攻击者可以在执行过程中动态选择查询，且这种安全性在实践中非常重要。

尽管这两种方案在安全性和效率上都更优，它们的设计意外地简单，为SSE技术的实际应用铺平了道路。

此外，文章还讨论了**多用户SSE**问题。与以往只允许数据所有者提交查询不同，新的方案允许除了数据所有者之外的多个用户提交搜索查询。文中正式定义了多用户SSE，并提出了一种高效的构造方法，其性能优于单纯使用访问控制机制的做法。

1 简介

主要关注如何在确保数据隐私的同时实现加密数据的搜索功能。

1. **私钥存储外包**：一些客户端由于资源或技术限制，选择将加密数据存储在外部服务上，但这也意味着他们失去了对加密数据的搜索能力。传统的私钥加密方法无法进行加密数据的选择性检索。
2. **可搜索加密**：为了弥补这一不足，研究者提出了可搜索加密技术，允许在加密数据中进行搜索。DARPA已将此领域列为平衡隐私与国家安全的关键技术之一。
3. **现有方法的局限性**：使用基于“无知RAM”（oblivious RAM）技术的方案能提供最高级别的隐私保护，但代价是需要大量的计算和通信资源，并且每次读写都需要多轮交互。已有的可搜索加密方案通过放宽隐私保护（如泄露访问模式）来提高效率。
4. **新定义和新方案**：作者提出了新的安全定义，区分了非自适应（non-adaptive）和自适应（adaptive）安全，并给出了两种高效的构造方案：

- 第一种是目前最高效的非自适应可搜索加密方案，服务器计算开销低，用户成

本为常数。

- 第二种方案实现了自适应安全，这是以前常规方案无法达到的。

5. **多用户设置**：此外，作者还扩展了可搜索加密问题到多用户环境，允许多个授权用户搜索同一文档集。

关于隐私搜索的不同模式

1. 私钥加密数据搜索 (Searching on Private-key Encrypted Data)

- 用户自行加密数据并可自由组织数据结构以便高效访问。
- 数据及附加数据结构加密后存储在服务器，只有持有私钥的人才能访问。
- 用户初次处理数据时的工作量与数据大小相当，但后续访问数据的工作量非常小。
- 可以完全隐藏用户的访问模式。

2. 公钥加密数据搜索 (Searching on Public-key Encrypted Data)

- 用户将数据加密后发送到服务器，且数据加密者和解密者可能不同。
- 典型应用中，用户发布公钥，多个发送者使用公钥将邮件发送到服务器，只有私钥持有者能够生成“陷阱门”进行关键词搜索。
- 原始公钥加密关键词搜索（PEKS）方案会泄露访问模式，但后续研究提供了能够隐藏访问模式的方案，尽管搜索时间开销为数据库大小的平方根，效率远低于私钥加密方案。
- 近年来提出了非对称高效可搜索加密（ESE），使得任何人都可以为索引添加词汇并生成陷阱门来搜索。虽然ESE方案实现了最佳搜索时间，但ESE方案是确定性的，提供的安全性保证比较弱。

在ESE（非对称高效可搜索加密）方案中，“确定性”指的是在给定相同的输入（例如相同的关键词或查询），系统每次都会生成相同的输出（例如相同的“陷阱门”或搜索结果）。换句话说，无论查询多少次，只要输入不变，生成的结果也不会变化。

这与非确定性方案相对，后者可能在每次查询时生成不同的结果或搜索路径，从而增强了隐私保护，因为它使得攻击者无法通过查询模式推测出更多信息。ESE的确定性特性意味着它在某些情况下可能提供较弱的隐私保护，因为相同的输入总是会导致相同的搜索行为，暴露了某些访问模式。

3. 私人信息检索（PIR） (Private Information Retrieval, PIR)

- PIR允许用户从包含未加密数据的服务器检索数据而不泄露访问模式，并且总通信量小于数据大小。
- 在关键词搜索场景下，PIR方案的工作量与数据库大小线性相关，因为如果不遍历所有数据项，服务器会知道未接触的项目不被用户关注。
- 尽管PIR方案允许多次查询和多用户共享工作，但其关键特点是数据始终未加密，区别于私钥加密和公钥加密的数据搜索。

总结：

这几种隐私搜索模型各有不同：

- **私钥加密数据搜索**：用户加密并控制数据存储，能完全隐藏访问模式，效率高。
- **公钥加密数据搜索**：允许公钥加密者与解密者不同，能够生成陷阱门进行搜索，但通常效率较低。
- **私人信息检索（PIR）**：数据始终未加密，访问模式通过线性工作量隐藏，但效率不如加密数据搜索方案。

这些模型的核心差异体现在数据是否加密、搜索效率和隐私保护的程度。

相关工作

1. Goldreich 和 Ostrovsky的工作：

- 他们提出了基于无知RAM（Oblivious RAM）的软件保护技术，但这一技术的轮次复杂度较高，影响了效率。

2. Song, Wagner 和 Perrig的工作：

- 为了降低无知RAM的复杂度，提出了一个为每个单词设计双层加密结构的方案。通过陷阱门，服务器可以去掉外层加密，并验证内层是否符合正确的形式。
- 然而，该方案存在一些局限：
 - 虽然它被证明是一个安全的加密方案，但并未证明是一个安全的可搜索加密方案；
 - 明文分布容易受到统计攻击；
 - 搜索时间是线性的，依赖于文档集合的长度。

3. Goh 和 Chang & Mitzenmacher的工作：

- 为了解决上述问题，Goh 和 Chang & Mitzenmacher提出了基于“索引”的方案，每

个文档有一个“索引”，服务器需要搜索这些索引，查询的工作量与文档集合的数量成比例。

- Goh 提出了索引安全性模型（如INDCKA和IND2-CKA），并使用布隆过滤器和伪随机函数来构造可搜索加密。
- Chang & Mitzenmacher 提出了与IND2-CKA类似的安全性模型，并确保陷阱门不泄露查询的单词信息。

4. 公钥加密与关键词搜索：

- 关键词搜索也已在公钥加密设置中考虑过。公钥加密方案允许任何人使用公钥向索引中添加词汇，但只有私钥持有者能生成陷阱门进行搜索。
- 公钥加密方案适用于不同的应用，但其效率较低，不如私钥加密方案高效。

5. 非对称高效可搜索加密（Asymmetric ESE）：

- ESE方案提供了与私钥方案相当的效率，但提供的安全性较弱。

6. 多用户可搜索加密（Multi-user SSE）：

- 本研究提出了多用户可搜索加密（Multi-user SSE）的概念，结合经典的公钥加密方案，能够实现与非对称ESE类似的功能，并且允许所有者撤销搜索权限。

总结：

该部分回顾了可搜索加密领域的相关工作，重点讨论了ORAM、Song等人提出的双层加密结构、Goh的索引安全模型、以及公钥加密和非对称ESE等方案。当前研究进一步扩展了多用户可搜索加密的概念，提供了改进的安全性和更灵活的权限管理。

作者的成果

1. 安全性定义的改进：

- 审查现有的可搜索加密（SSE）安全性定义，包括IND2-CKA 和 Chang & Mitzenmacher 的基于仿真定义，指出了其局限性。
- 提出新的不可区分性和基于仿真的安全定义，确保对**索引**和**陷阱门**的安全性，并证明它们是等价的。

2. 引入新的对抗模型：

- 提出两种新的对抗模型：

- **非自适应模型 (Non-adaptive)**：只考虑没有参考先前查询结果和陷阱门的对手。
- **自适应模型 (Adaptive)**：对手可以根据先前获得的查询结果和陷阱门来选择新的查询。
- 所有现有SSE工作（除无知RAM外）都属于非自适应设置。该研究通过新的定义解决了在自然使用中的安全性问题。

3. 提出了两个安全构造：

- **SSE-1**：仅在非自适应模型下安全，最有效的SSE构造之一。它在一次通信轮次内执行搜索，服务器工作量与查询单词出现的文档数量成正比，客户端存储恒定，服务器存储与文档集合大小成线性关系。
- **SSE-2**：在自适应对手模型下安全，虽然通信开销更大，服务器存储要求更高，但与现有方案相比，它具有更高的效率和安全性。

4. 多用户设置的扩展：

- 扩展了SSE模型到**多用户设置**，即数据所有者可以允许多个用户查询其文档集合，且可以通过授予和撤销搜索权限来控制访问。
- 提出了一个高效的多用户构造，能够在没有认证的情况下工作，优于传统的访问控制机制。

5. 服务器模型和安全性：

- 论文假设服务器为**诚实但好奇**，并且提到可以通过内存检查和通用论证等技术，使得方案能够防御恶意服务器，但需要额外开销。这个扩展将在论文的完整版中进一步讨论。

2 预备知识

1. 词典和文档集合：

- Δ 是包含 d 个单词的字典， 2^Δ 是所有可能文档的集合。
- $D \subseteq 2^\Delta$ 是文档集合，包含 n 个文档，表示为 $D = (D_1, \dots, D_n)$ 。
- $\text{id}(D)$ 表示文档 D 的唯一标识符（如内存位置）。
- $D(w)$ 表示包含单词 w 的所有文档的标识符列表。

2. 访问模式和搜索模式：

- **访问模式**：指包含每个单词 w 的所有文档的标识符列表（即 $D(w_1), \dots, D(w_n)$ ）。

))。

- **搜索模式**：通过判断是否对同一个单词进行了两次查询，可以推导出的信息。

3. 符号定义:

- $x \leftarrow \mathcal{X}$ 表示从分布 $\mathcal{X}$ 中采样一个元素 x 。
- $x \stackrel{R}{\leftarrow} X$ 表示从集合 X 中均匀采样一个元素 x 。
- $x \leftarrow \mathcal{A}$ 表示算法 $\mathcal{A}$ 输出的结果是 x 。
- $\parallel$ 表示字符串连接。
- **可忽略函数**：如果对每个多项式 $p(k)$ 和足够大的 k ，满足 $v(k) < 1/p(k)$ ，则 $v(k)$ 称为可忽略函数。

4. 模型:

- **单用户可搜索加密 (SSE)**：一个用户想要将加密的文档集合存储在诚实但好奇的服务器上，同时仍能对这些文档进行搜索。
- **多用户可搜索加密 (SSE)**：包括一个信任的所有者 O ，一个诚实但好奇的服务器 S 和一组用户 N ， O 授权或撤销 N 中某些用户的搜索权限。

5. 基本加密原语:

- **对称加密方案**：包括三个多项式时间算法 (G, E, D)，分别用于生成密钥、加密和解密。
 - G ：生成一个密钥 K 。
 - E ：使用密钥 K 加密消息 m ，返回密文 c 。
 - D ：使用密钥 K 解密密文 c ，返回明文 m 。
- **安全性**：对称加密方案被认为是安全的，如果密文不泄露关于明文的任何信息，即使是能够适应性查询加密和解密神谕的攻击者。
- **伪随机函数 (PRF) 和置换 (PRP)**：这些是多项式时间计算的函数，无法被任何概率多项式时间的攻击者区分为随机函数。

3 重新审视SSE定义

定义 3.1 (可搜索对称加密方案, SSE)

一个可搜索对称加密方案 (SSE) 由四个多项式时间算法组成: **Keygen**、**BuildIndex**、**Trapdoor** 和 **Search**，其满足以下条件:

1. $\text{Keygen}(1^k)$: 一个概率算法, 用于密钥生成。它接收一个安全参数 k , 并返回一个秘密密钥 K , 其中 K 的长度是多项式级别的。
2. $\text{BuildIndex}(K, \mathcal{D})$: 一个 (可能是概率) 算法, 用于生成索引。它接收一个秘密密钥 (K) 和一个多项式大小的文档集合 ($\mathcal{D}$) 作为输入, 返回一个索引 (I), 其中 (I) 的长度也是多项式级别的。
3. $\text{Trapdoor}(K, w)$: 一个算法, 用于生成陷门。它接收一个秘密密钥 K 和一个查询词 w , 并返回一个陷门 T_w 。
4. $\text{Search}(I, T_w)$: 一个由服务器执行的搜索算法。它接收一个索引 I 和一个陷门 T_w 作为输入, 返回包含查询词 w 的文档的标识符集合 $D(w)$ 。

这个方案允许在保密的前提下, 基于加密索引搜索包含特定单词的文档集合。

建立准确的安全可搜索加密的定义

1. SSE的安全性定义问题:

- 到目前为止, SSE (可搜索加密) 的安全性定义仍然不明确且难以达成一致。
- 尽管可以参考ORAM中的通用定义, 但研究者尝试通过揭示一些信息来提高效率, 这使得安全性定义的明确性更加复杂。

2. 传统的SSE安全定义:

- 文献中通常将SSE的安全性定义为, 除了搜索结果 (即返回的文档标识符) 之外, 不应泄露任何其他信息。
- 这种定义本身存在问题, 因为许多现有的SSE方案 (除了ORAM) 都泄露了搜索模式。

3. 搜索模式的泄露:

- 现有的SSE方案 (如文献[30, 17, 13]中的方案) 泄露了比预期更多的信息, 特别是“搜索模式”。
- “搜索模式”指的是通过比较两次搜索是否查询了同一个词, 推断出的一些信息。

4. 更准确的安全定义:

- 作者认为, SSE的安全性应当定义为: 除了搜索结果和搜索模式之外, 不应泄露更多的信息。
- 搜索模式是指从知道是否对同一个词执行了两个搜索所能推导出的任何信息。

5. 现有研究的局限性：

- 除了ORAM之外，现有的SSE方案普遍会泄露用户的搜索模式，而这一点在安全性定义中尚未被充分考虑或解决。

先前SSE定义的局限性

1. 攻击者能力的定义问题：

- 在Song, Wagner和Perrig的工作中，虽然他们证明了他们的构造是安全的，但其隐含的安全定义类似于传统加密方案，其中攻击者不能执行搜索操作。这种定义未能充分考虑攻击者可能的行为。
- Goh提出了“对选择关键字攻击的不可区分性”（IND2-CKA）的定义，这种定义解决了部分问题，即攻击者（服务器）即使能访问索引，也无法获得关于文档的部分信息，除非是通过客户端提供的陷门生成的。
- IND2-CKA的定义实际用于更广泛的问题——安全索引，它不仅限于搜索加密，且并非为SSE专门设计。

2. IND2-CKA与SSE的适用性问题：

- 尽管很多SSE研究使用IND2-CKA作为安全定义，但Goh明确表示，IND2-CKA并未要求陷门的安全性，而陷门安全对于搜索加密而言至关重要。因此，单独使用IND2-CKA无法全面保障SSE的安全性。

3. 索引和陷门安全的内在联系：

- 即使单独考虑陷门，也有可能它们在与索引结合时泄露更多信息。Goh的工作和文献[14]展示了这种情况：某些SSE方案中的陷门本身可能不会泄露关于查询词的信息，但与索引结合后，攻击者可以恢复整个查询词。因此，索引和陷门的安全性是密切相关的。

4. Chang和Mitzenmacher的模拟定义：

- 为了解决陷门和索引安全问题，Chang和Mitzenmacher提出了一个模拟基础的定义，旨在保障索引和陷门的隐私。他们借鉴了传统加密方案中语义安全的定义，要求任何从索引和陷门中计算出的信息，都能从搜索结果中推导出来。
- 然而，这种定义在SSE中的适用性有限，因为需要明确搜索查询是如何生成的，特别是是否是自适应的（基于先前搜索结果生成的查询）还是非自适应的（不依赖于先前结果）。这一区分非常重要，因为它直接影响安全性定义的隐私保障。

- **非自适应**的定义只对一次性生成所有查询的客户端提供保障，而**自适应**定义则能对那些根据先前查询结果生成新查询的客户端提供隐私保障。

5. 定义的局限性：

- Chang和Mitzenmacher的定义存在问题，因为它不仅是非自适应的，而且任何SSE方案（即使是不安全的）都能满足这个定义，因此该定义在保障SSE安全性方面是不充分的。

我们的安全定义

1. 历史与视图定义：

- **History**：指的是客户端与服务器之间的互动，包括文档集合和客户端希望搜索的词汇。
- **view**：是攻击者实际能看到的消息，包括索引（文档集合的索引）、（查询词的）陷门以及一些公共信息（如文档数量和加密文本）。
- 在安全设计中，视图应仅揭示搜索结果和搜索模式，不泄露关于文档和查询词的其他信息。

2. 泄露信息的范围：Trace：

- Trace指的是攻击者可获知的有关历史的有限信息，包括搜索结果和搜索模式

3. 非自适应不可区分性安全定义：

- **定义3.2**：一种SSE方案在非自适应不可区分的意义上是安全的，如果对于任何两个具有相同长度和trace的history的对抗性构造，没有任何（概率多项式时间）对手能够以非微不足道地优于 $1/2$ 的概率区分视图与另一个视图。
- **非自适应不可区分性安全**要求，对于任意两种具有相同长度和trace的history，攻击者无法以非微不足道的概率区分它们的视图。此定义假设攻击者在开始时一次性生成所有历史，而不是根据先前的信息做出选择。

具体来说，这里的“非微不足道的概率”是指攻击者不能在超出随机猜测的概率范围内区分两者。例如，若攻击者只能以大约50%的概率正确区分这两个历史（也就是和随机猜测一样），那么可以认为这个安全性条件被满足了。

4. 自适应不可区分性安全定义：

- **定义3.3**：一个SSE方案在自适应不可区分的意义上是安全的，如果对于任何两

个具有相同长度和trace的自适应构造的history，任何（概率多项式时间）对手都不能以非微不足道地优于 $1/2$ 的概率区分一个历史的视图与另一个历史的视图。

- **自适应不可区分性安全**允许攻击者根据先前查询的结果（包括索引和陷门）逐步构造查询词。攻击者能根据已获得的信息进行更复杂的攻击，但定义要求在此情况下，攻击者也无法区分两种历史的视图，且概率上与随机猜测相似。

5. 模拟基础的安全定义：

- 提出了模拟基础的安全定义（类似语义安全），通过构造一个模拟器，允许其根据trace生成视图中可以计算的所有信息。作者还证明了**非自适应和自适应不可区分性安全与非自适应和自适应语义安全**是等价的。
- **定理3.4**：SSE的非自适应（自适应）不可区分安全性等价于SSE的非自适应（自适应）语义安全性。

6. 结论：

- 这些改进的定义提供了对SSE方案的更严密的安全保障，尤其是在处理适应性攻击和信息泄漏时，确保了方案的健壮性和合理性。

4 高效和安全的SSE

1. **目标**：本节介绍了高效的可搜索加密（SSE）构造，并根据第三节中提出的安全定义对其进行了安全性表述。

2. 额外符号和数据结构：

- 设 $\Delta', \Delta \subseteq \Delta'$ 为文档集合 $\mathcal{D}$ 中存在的不同单词的集合。
- 假设 Δ 中的单词最多可以用 p 位表示。
- 记 $\mathcal{D}(w)$ 为文档集合 $\mathcal{D}$ 中包含单词 w 的文档标识符集合，按字典序排列。

3. 数据结构：

- 使用了数组、链表和查找表等数据结构。
- 数组 A 中的第 i 个元素记为 $A[i]$ ，元素 x 在数组中的地址记为 $\text{addr}(A(x))$ 。若 $A[i] = x$ ，则 $\text{addr}(A(x)) = i$ 。
- 链表 L 存储在数组 A 中，每个节点 N_i 的格式为 $\langle v_i; \text{addr}(A(N_i + 1)) \rangle$ ，其中 v_i 是一个任意字符串， $\text{addr}(A(N_i + 1))$ 是链表中下一个节点的内存地址。

4.1 高效的SSE构建

1. 一轮非自适应SSE构造:

- 该构造为文档集合 $\mathcal{D}$ 关联一个单一的索引 $\mathcal{I}$, $\mathcal{I}$ 包含两个数据结构:
 - **数组 A**: 存储每个单词 $w \in \Delta'$ 对应的加密后的文档标识符集合 $\mathcal{D}(w)$ 。
 - **查找表 T**: 包含帮助定位并解密 A 中相关元素的信息。

2. 链表存储与加密:

- 为每个单词 $w_i \in \Delta'$ 创建链表 L_i , 链表节点为文档标识符。
- 将所有链表 L_i 的节点按随机顺序“打乱”, 加密后存储到数组 A 中。
- 每个链表节点在加密前增加关于下一个节点在 A 中位置的索引和加密密钥。
- 通过这种方式, 服务器能够根据位置和密钥解密链表中的所有节点。

3. 查找表 T 的构建:

- 查找表 T 中每个条目对应一个单词 $w_i \in \Delta$, 包含一对 $\langle \text{地址}, \text{值} \rangle$ 。
- "值"字段包含链表 L_i 首节点的索引和密钥, 值本身也被加密。
- "地址"字段用于定位 T 中的条目。
- 查找表 T 使用间接寻址管理。

4. 查询过程:

- 用户根据未加密的文档集合 $\mathcal{D}$ 计算并存储数组 A 和查找表 T, 并将其与加密后的 $\mathcal{D}$ 一同存储到服务器。
- 当用户想检索包含某单词 w_i 的文档时, 计算对应条目的解密密钥和地址, 并发送给服务器。
- 服务器解密并定位对应条目, 获取链表 L_i 的首节点索引和密钥, 进而解密 L_i 中所有节点, 得到文档标识符 $\mathcal{D}(w_i)$ 。

稀疏表的高效存储和访问

1. 间接寻址与查找表管理:

- 在查找表 T 中, 每个条目是一个元组 $\langle \text{address}, \text{value} \rangle$, 其中 **address** 用作虚拟地址来定位条目。
- 虚拟地址的域大小为指数级 (即 $\{0, 1\}^p$), 但查找表中的条目数量是多项式

级别，因此使用的虚拟地址数量大约是 $\text{poly}(p)$ 。

2. 查找表结构：

- 如果查找表 T 的地址字段来自 $\{0, 1\}^p$ ，值字段来自 $\{0, 1\}^v$ ，且表中最多有 m 个条目，则 T 被称为一个 $(\{0, 1\}^p \times \{0, 1\}^v \times m)$ 的查找表。

3. 高效存储与查找：

- 为了高效存储查找表 T ，使用 FKS 字典（一种稀疏表存储的高效数据结构）。
- 使用 FKS 字典，能以 $O(|\text{Addr}|)$ 存储和 $O(1)$ 查找时间来管理虚拟地址集合 Addr 。
- 对于一个虚拟地址 A ，能在常数时间内判断 A 是否属于 Addr ，如果属于，则返回与之关联的值。对于不在 Addr 中的地址，认为它是未定义的。

SSE-1 的详细信息

1. 安全参数与加密方案：

- 设定安全参数为 k 和 l ，使用语义安全的对称加密方案 (G, E, D) ，其中 E 为加密函数， D 为解密函数。
- 使用伪随机函数 f 和两个伪随机置换 π 和 ψ ，具有特定的参数。

2. 文档集合的管理：

- 设定文档集合的总大小为 m ，并将其存储在数组 A 中，数组 A 的大小为 m 。
- 使用间接寻址管理的查找表 T ，其大小为 $(\{0, 1\}^p \times \{0, 1\}^{l+\log_2(m)} \times |\Delta|)'$ 。
- 对于每个词 w_i ，对应一个链表 L_i ，存储包含该词的文档的标识符。

3. 防止信息泄露：

- 为避免泄露文档中某词的重复出现，设置数组 A 的大小为 m ，并用随机值填充未用到的空间。

如果每个文档 $\mathcal{D}_j \in \mathcal{D}$ 中的每个单词都只出现一次（即文档中没有重复的词），那么 m 和 m' 就是相等的，因为每个单词只会出现一次，所以 m 就是所有文档中的单词总数，而 m' 则是所有链表 L_i 中的元素总数。

换句话说, $m = m'$ 是在没有重复词的情况下的理想情况。

- 查找表 T 也使用随机填充, 确保条目数量等于词典大小 $|\Delta|$, 避免泄露文档中不同词的数量。

4. SSE-1 构造和安全性:

- 构造了一个非自适应安全的 SSE 方案 $SSE-1 = (Keygen, BuildIndex, Trapdoor, Search)$, 并提供了安全证明。
- **定理4.1:** SSE-1 是一个非自适应安全的 SSE 方案。
- 该方案的查询只需一个回合, 并且消息大小为 $O(1)$ 。

算法分析

1. Keygen 和 BuildIndex

- Keygen:
 - 生成随机密钥 s, y, z , 以及加密参数 ℓ 。
 - 输出密钥 $K = (s, y, z, 1^\ell)$ 。
- BuildIndex (构建索引): 主要步骤包括 **初始化**、**构建数组 A** 和 **构建查找表 T**:

1.1 初始化

- 扫描文档集合 $\mathcal{D}$, 生成唯一单词集合 Δ' 。
- 对每个单词 $w \in \Delta'$, 建立 $\mathcal{D}(w)$, 即包含 w 的文档标识符集合。

1.2 构建数组 A

- **目标:** 将每个单词对应的文档标识符列表 L_i 组织成链表, 并加密后存储在数组 A 中。
- **构建链表 L_i :**
 - 对每个单词 $w_i \in \Delta'$, 构建链表 L_i , 节点 $N_{i,j}$ 包含:
 - a. **文档标识符** $id(D_{i,j})$: 表示包含单词 w_i 的第 j 个文档。
 - b. **随机生成的加密密钥** $\kappa_{i,j}$ 。
 - c. **下一个节点的地址**, 通过伪随机置换 $\psi_s(ctr + 1)$ 计算。
 - 每个节点 $N_{i,j}$ 用加密函数 E 加密后存储在数组 A 的相应位置。
 - **链表终止处理:** 对最后一个节点, 将“下一个节点地址”设置为 **NULL**。

- **填充数组：**

- 计算 $m' = \sum_{w_i \in \Delta'} |\mathcal{D}(w_i)|$ （所有链表节点的总数）。
- 如果 $m' < m$ ，则用随机值填充数组 A 的剩余位置，避免泄露文档单词的重复信息。

1.3 构建查找表 T

- **目标：**构建一个查找表 T，用于存储每个单词的链表起始节点地址和解密密钥。

- **步骤：**

- 对每个 $w_i \in \Delta'$ ：
 - a. 计算 $\text{value} = (\text{addr}(A(N_{i,1})) \parallel \kappa_{i,0}) \oplus f_y(w_i)$ ，其中 f_y 是伪随机函数。
 - b. 使用 $\pi_z(w_i)$ 作为虚拟地址，将 value 存入 T。
 - **填充：**如果 $\Delta' < |\Delta|$ ，将剩余条目随机填充，避免泄露文档中唯一单词的数量。
-

2. Trapdoor 和 Search

- **Trapdoor：**

- 输入查询词 w ，计算查询陷门 $T_w = (\pi_z(w), f_y(w))$ 。
- $\pi_z(w)$ ：定位查找表 T 中对应条目的地址。
- $f_y(w)$ ：用于恢复链表的起始地址和解密密钥。

- **Search：**

- i. 根据陷门 T_w 检索查找表 T 中的条目 $\theta = T[\gamma]$ ，提取链表的起始地址 α 和解密密钥 κ 。
 - ii. 从数组 A 中，解密链表的第一个节点，并依次解密后续节点，得到所有文档标识符。
 - iii. 输出链表 L 中包含的文档标识符，即 $\mathcal{D}(w)$ 。
-

3. 主要特性与分析

1. 安全性:

- 本方案是**非自适应安全**的（根据图中的说明和理论证明）。
- 它隐藏了搜索查询的**结果和模式**，只泄露文档集合的加密总大小和文档数量。

2. 效率:

- **查询**：只需要**一轮通信**。
- **计算开销**：
 - 用户生成陷门的时间复杂度为 $O(1)$ 。
 - 服务器检索文档的时间复杂度与链表 $|L_i|$ 的长度成正比，即 $O(|\mathcal{D}(w)|)$ 。
- **存储开销**：
 - 服务器存储的加密文档集合为 $O(m)$ 。
 - 数组 A 的大小为 $O(m)$ ，查找表 T 的大小为 $O(|\Delta|)$ 。

3. 填充机制:

- 通过随机填充数组 A 和查找表 T ，避免泄露关于文档中单词重复次数和唯一单词数量的信息。

关键点:

- **SSE-1** 方案安全且高效，避免泄露文档中某些信息，支持高效存储与查询。
- 使用间接寻址方法存储查找表，减少信息泄露。
- 方案的存储需求和查询效率都保持在低复杂度水平。

自适应SSE安全性

1. 挑战与解决方案

SSE-1 难以证明针对自适应对手的安全性，因为需要提前构建与未来查询一致的对手视图。SSE-2 通过为每个单词 w 的实例生成标签族 F_w （如“coin1”，“coin2”等）来规避这一问题。搜索 w 等价于搜索其标签族中的所有标签，每个标签只与一个文档相关联，使得对手视图易于模拟。

2. 构建索引

- 索引 I 由查找表 T 组成。
- 每个单词标签族的元素都有一条索引条目，存储包含该标签的文档标识符。
- 为隐藏每个文档中的不同单词数，对 T 进行填充，确保每个文档标识符出现在相同数量的条目中。

3. 查询过程

- 用户搜索单词 w 时，需要检索 w 的标签族 F_w 中的所有标签。
- 为提高效率，可以在每个条目中存储 $|D(w)|$ 的加密值，用户通过第一次查询获取 $|D(w)|$ ，进而派生剩余标签的陷门。

4. 性能与开销

- SSE-2 和 SSE-1 均在**一轮通信**内完成查询，服务器的计算量与匹配文档数量 $O(|D(w)|)$ 成正比。
- 用户的计算和存储开销为 $O(1)$ 。
- 服务器存储量较 SSE-1 增加了 $\max$ 倍（其中 $\max$ 为最大文档大小）。
- 通信开销与 $\max$ 成正比，但可通过优化降低。

5. 安全性

在理论证明中，SSE-2 达到了针对**自适应对手**的安全性。

定理 4.2. SSE-2 是一个自适应安全的 SSE 方案。

算法分析

1. Keygen (密钥生成)

- 生成一个随机密钥 s ，输出 $K = s$ 。
-

2. BuildIndex (索引构建)

构建索引 I ，具体过程如下：

步骤 2.1: 初始化

- 遍历文档集合 $\mathcal{D}$ ，建立所有**不同单词**组成的集合 Δ' 。
- 对于每个单词 $w \in \Delta'$ ，建立其文档标识符集合 $\mathcal{D}(w)$ 。

步骤 2.2: 构建查找表 T

a) 对于每个单词 $w_i \in \Delta'$ 和 $1 \leq j \leq |\mathcal{D}(w_i)|$:

- 生成标签 $\pi_s(w_i||j)$, 其中 $w_i||j$ 表示单词 w_i 和索引 j 的拼接。
- 存储 T 中的值: $value = id(D_{i,j})$, 即单词 w_i 在文档 $D_{i,j}$ 中的标识符。
- 设置 $T[\pi_s(w_i||j)] = value$ 。

b) 填充查找表:

- 计算 $m' = \sum_{w_i \in \Delta'} |\mathcal{D}(w_i)|$, 即所有单词实例的总数。
- 如果 $m' < m$ (m 为查找表的总大小), 则将剩余 $(m - m')$ 个条目填充为随机值, 确保:
 - 每个文档 $D \in \mathcal{D}$ 的 `value` 至少出现 m 次。
 - 填充值的地址字段也设置为随机。

目的: 通过填充, 隐藏每个文档中不同单词的数量, 防止信息泄露。

步骤 2.3 Output (索引输出)

输出构建好的索引 $\mathcal{I} = T$ 。

4. Trapdoor (生成陷门)

- 给定一个查询单词 w , 生成对应的陷门 T_w :
 $T_w = (\pi_s(w||1), \dots, \pi_s(w||\max))$, 即生成 w 的所有标签 $w||1, w||2, \dots$ 到最大文档大小 $\max$ 。

5. Search (搜索)

- 用户将陷门 T_w 发送给服务器。
- 服务器按以下步骤执行:
 - i. **检索:** 对每个标签 $\pi_s(w||i)$ ($1 \leq i \leq \max$), 从查找表 T 中检索 $id = T[\pi_s(w||i)]$ 。
 - ii. **输出:** 返回对应的文档标识符 id 。

5 多用户SSE

1. 背景与问题

在多用户可搜索加密（MSSE）场景中，用户拥有一个文档集合，但多个授权用户可以对该集合执行搜索查询。核心挑战包括：

- **管理访问权限**：添加或撤销用户的权限。
 - **保持服务器端的隐私性**：服务器可以执行查询，但不能学习未授权信息。
-

2. 多用户 SSE 定义

一个 MSSE 方案由六个多项式时间算法组成：

1. **MKeygen**：生成密钥 K_O （由数据拥有者运行）。
 2. **MBuildIndex**：基于文档集合 D 和密钥 K_O 构建索引 $\mathcal{I}$ 。
 3. **AddUser**：添加用户 U ，分配用户密钥 K_U 。
 4. **RevokeUser**：撤销用户 U 的权限。
 5. **MTrapdoor**：用户使用其密钥 K_U 生成查询陷门 $T_{U,w}$ 。
 6. **MSearch**：服务器基于索引 $\mathcal{I}$ 和陷门 $T_{U,w}$ 执行搜索。如果用户 U 有权限，则返回结果 $D(w)$ ；否则返回空 $\perp$ 。
-

1. MKeygen（密钥生成）

- 输入：安全参数 1^k 和 1^ℓ 。
- 过程：生成**单用户 SSE 密钥** K^s 和**伪随机置换密钥** r 。
- 输出：文档拥有者的主密钥 $K_O = (K^s, r)$ 。

解释：

- K^s ：用于单用户 SSE 加密索引和生成陷门。
 - r ：用于伪随机置换 ϕ ，保障用户查询的安全性和撤销机制。
-

2. MBuildIndex（索引构建）

- 输入: K_O (拥有者密钥) 和文档集合 $\mathcal{D}$ 。
- 过程:
 - 运行单用户 SSE 构建索引 I^s 。
 - 初始化广播加密 (BE) 系统。
 - 设置初始撤销用户集合 $R = \emptyset$ 。
 - 将密钥 r 和广播加密的结果 $\mathcal{E}_N^{\text{BE}}(r)$ 发送给服务器。
- 输出: 服务器持有的索引 $I^m = I^s$ 。

解释:

- 索引 I^s 是单用户 SSE 的索引结构。
 - 广播加密 (BE) 确保 r 只能被未撤销的用户恢复。
-

3. AddUser (添加用户)

- 输入: K_O (拥有者密钥) 和新用户 U 。
- 过程:
 - 将用户密钥 $K_U = (K^s, r)$ 发送给用户 U 。
 - 发送与广播加密相关的长效密钥 (long-lived BE secrets) 给用户。

解释:

- 用户获取 K^s (用于生成陷门) 和当前的 r (用于伪随机置换 ϕ) 。
 - 长效密钥允许用户解密广播加密发送的内容, 支持密钥更新。
-

4. RevokeUser (撤销用户)

- 输入: K_O 和被撤销用户 U 。
- 过程:
 - 生成新的密钥 r' 。
 - 将新密钥 r' 和更新后的广播加密结果 $\mathcal{E}_{N \setminus R}^{\text{BE}}(r')$ 发送给服务器。
 - 服务器用 r' 覆盖旧的 r 。

解释：

- 通过广播加密，只有未被撤销的用户可以恢复新的 r' ，而被撤销用户无法生成有效查询陷门。
-

5. MTrapdoor (生成陷门)

- 输入：用户密钥 K_U 和搜索关键词 w 。
- 过程：
 - 生成单用户 SSE 陷门 $T_{U,w}^s$ 。
 - 从服务器恢复当前密钥 r (通过广播加密)。
 - 使用伪随机置换 ϕ 计算 $T_{U,w}^m = \phi_r(T_w^s)$ 。
- 输出： $T_{U,w}^m$ (经过伪随机置换的陷门)。

解释：

- ϕ_r 为关键词陷门增加了一层加密保护。
 - 用户需要广播加密中的长效密钥来解密服务器的 r 。
-

6. MSearch (服务器搜索)

- 输入：索引 I^m 和陷门 $T_{U,w}^m$ 。
- 过程：
 - 服务器计算 $T_w^s = \phi_r^{-1}(T_{U,w}^m)$ (使用当前密钥 r 的逆置换)。
 - 验证 T_w^s 是否为有效的虚拟地址。
 - 如果有效，则执行单用户 SSE 的搜索操作 $\text{Search}(I^m, T_w^s)$ ，并返回结果；否则返回 $\perp$ 。

解释：

- ϕ_r^{-1} 将 $T_{U,w}^m$ 转换回单用户 SSE 陷门 T_w^s 。
 - 服务器通过简单计算 ϕ 和其逆置换确保用户查询的合法性。
-

3. 安全性要求

- **与单用户安全性的关系**：多用户方案的安全性可以简化为底层单用户 SSE 方案的语义安全性，因为主要关注的是服务器端的安全威胁。
- **撤销安全性**：一旦用户被撤销，该用户必须无法继续对文档集合执行搜索。

4. 正确性

一个 MSSE 方案是**正确的**，如果对任意单词 w ，任意授权用户 U ，服务器能够正确返回文档集合中包含 w 的结果：

$$\Pr[MSearch(\mathcal{I}_D, T_{U,w}) = D(w) : U \in G] = 1$$

5. 方案构建

- **构建方法**：将单用户 SSE 方案与**广播加密 (BE)** 方案结合。
- **广播加密 (BE)**：允许加密一条消息给一组动态变化的用户集合 G 。
 - 每个用户持有一组**长期密钥**，用于解密被加密的消息。
 - 被撤销的用户无法解密新消息，确保安全性。
- **MSSE 实现**：使用广播加密高效管理用户的添加和撤销，通过 BE 技术为用户分发陷门密钥，确保只有当前授权的用户可以执行搜索操作。

多用户可搜索加密 (MSSE) 方案的详细构建思路，关键点总结如下：

1. 方案概述

- **基本操作**：授权用户 U 生成单用户陷门 $T_{U,w}$ ，然后用一个密钥 r 对其应用**伪随机置换** ϕ ，得到 $\phi_r(T_{U,w})$ ，并将其发送给服务器。
- **服务器端处理**：服务器接收到 $\phi_r(T_{U,w})$ 后，使用 r 的逆置换 ϕ_r^{-1} 还原陷门 $T_{U,w}$ 。
- **密钥管理**：

- 当前密钥 r 由文档拥有者、授权用户和服务器共同持有。
 - 当用户被撤销时，拥有者生成新的 r ，并通过**广播加密 (BE)** 将新密钥分发给未撤销的用户。
 - 被撤销用户无法获取新密钥 r ，因此无法生成有效的陷门。
-

2. 安全性与机制

- **防止被撤销用户访问**：即使撤销用户发送查询，服务器在应用 ϕ_r^{-1} 后，其陷门无法验证为有效。
 - **安全依赖**：
 - BE 提供密钥分发的安全性，确保被撤销用户无法协作恢复 r 。
 - 伪随机置换 ϕ 提供额外安全层，防止未授权用户的成功查询。
 - 假设服务器是**诚实但好奇**，即只执行查询，但不主动篡改数据或攻击系统。
-

3. 实现细节

- MSSE 方案基于单用户 SSE 和广播加密 (BE) 构建。
 - 伪随机置换 ϕ 的输入输出长度由安全参数和文档集合大小决定。
 - **服务器效率**：服务器仅需评估 ϕ 和其逆置换 ϕ^{-1} ，无需更复杂的身份验证机制。
-

4. 结论

该方案通过**伪随机置换 ϕ** 和**广播加密**高效地管理用户权限，特别是在用户撤销的场景下，保证了安全性和效率。服务器在接收陷门时只需简单计算 ϕ 和 ϕ^{-1} ，避免了更复杂的访问控制协议。